

//! 24 – Dynamic Programming – Longest Common Subsequence –
Algorithm – Code

```
const text_01 = "HELLOWORLD";  
const text_02 = "OHELOD";
```

```
//& 1) Calculate LCS value
```

```
//? Define the length of the two texts:
```

```
// n = text_01.length  
// m = text_02.length
```

```
//? Trick to define rows and columns of zeros
```

```
text_01 = " " + text_01;  
text_02 = " " + text_02;
```

```
//? Define array for the results
```

```
//* const dp = []
```

```
//* dp[i][j] is the cell value
```

```
//? Define nested for loop to loop over rows and columns
```

```
//^ look up the slide
```

```
//^ outward loop: to loop over text_02 (rows)
```

```
for (let i = 0; i <= m; i++) {}
```

```
//^ inner loop: to loop over text_1 (columns)
```

```
for (let j = 0; j <= n; j++) {}
```

```
//? set conditions inside inner loop to:
```

```
// define rows and columns of zeros
```

```
if (i == 0 || j == 0) {  
    dp[i][j] = 0;  
}
```

```

// if character match
if (text_02[i] === text_01[j])
    dp[i][j] = 1 + dp[i - 1][j - 1]; /* 1 + top left corner
// if not match
else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]); /* Max(top cell, left cell)

//? Print the LCS value
console.log(dp[m][n]);

//& 2) Get LCS characters

/* start from the bottom right
/* dp[m][n]
/* if (dp[m][n] > dp[m][n - 1])      :
/*   if (dp[m][n] === dp[m - 1][n])
/*       the current inherited from top (not match)
/*       move to top row

/*   (dp[m - 1][n])
/*   else
/*       current value is origin of match
/*       add the char to the solution
/*       move to the top row
/*       move to left column
/* move to left column

//? define empty string

let str = "";
let i = m;
let j = n;

/* loop backwardly over both rows and columns
while (i > 0 && j > 0) {
    if (dp[i][j] > dp[i][j - 1]) {
        if (dp[i][j] === dp[i - 1][j]) {
            i--; // move to top row
        } else {
            str = text_02[i] + str; // add the char to the solution
            i--;

```

```

        j--;
    }
    } else {
        j--; // move to left column
    }
}

```

///
print the string

console.log(str);

i don't understand the part of get: Get LCS characters, i can't understand the logic, revise it, check it if correct and explain it clearly with illustration and tracing

Let's break down the **"Get LCS characters"** (backtracking) part of the algorithm. In this phase, we already have a fully filled DP table where each cell `dp[i][j]` tells us the length of the longest common subsequence (LCS) between:

- The first `i` characters of `text_02` (rows)
- The first `j` characters of `text_01` (columns)

Now, our goal is to trace back from `dp[m][n]` (bottom-right) to reconstruct the subsequence (in reverse order) that formed the LCS.

The Standard Backtracking Concept

The **standard backtracking** procedure for LCS is as follows:

1. **Start at the bottom-right corner:** Let `i = m` (last row) and `j = n` (last column).
2. **While both indices are positive:**
 - **If the characters match:** That is, if `text_02[i] === text_01[j]`, then this character is part of the LCS.
 - **Action:** Prepend this character to your result string, then move diagonally (i.e., `i--` and `j--`).
 - **If the characters do not match:** Compare the two candidates:
 - **If** `dp[i-1][j] > dp[i][j-1]`: The LCS came from the cell above.
 - **Action:** Move up by doing `i--`.
 - **Else:** The LCS came from the left cell.
 - **Action:** Move left by doing `j--`.

In pseudocode, it looks like this:

Javascript

Copy

```
let result = "";
while(i > 0 && j > 0) {
  if (text_02[i] === text_01[j]) {
    result = text_02[i] + result; // or text_01[j], they are equal in a match
    i--;
    j--;
  } else if (dp[i-1][j] > dp[i][j-1]) {
    i--;
  } else {
    j--;
  }
}
```

The Provided Code's Approach

The provided code uses a slightly different way to decide whether a cell comes from a match or simply from inheriting a previous computed value. Here is the snippet:

Javascript

Copy

```
let str = "";
let i = m; // row index (for text_02)
let j = n; // column index (for text_01)
while (i > 0 && j > 0) {
  if (dp[i][j] > dp[i][j - 1]) {
    if (dp[i][j] === dp[i - 1][j]) {
      i--; // move up: the value was inherited from above, so no match here
    } else {
      str = text_02[i] + str; // a match occurred - add this character
      i--;
      j--;
    }
  } else {
    j--; // move left: the value on the left is equal or higher, meaning the current
  }
}
```

Let's explain each part:

1. `if (dp[i][j] > dp[i][j - 1]) { ... }`

- **Interpretation:** This checks if the current cell's LCS length is *greater* than the cell immediately to its left.
- **Why?** If it were merely inherited from the left (i.e., no new match was added when moving from `dp[i][j-1]` to `dp[i][j]`), you would expect them to be equal.
- **Implication:** A greater value means either a match added by moving diagonally or that the "advantage" came by including the current row's character.

2. Nested Condition: `if (dp[i][j] == dp[i - 1][j])`

- **Interpretation:** Even if the current cell is greater than the left one, it might still be equal to the cell above.
- **Why?** If `dp[i][j]` didn't increase compared to `dp[i - 1][j]`, it indicates that moving from the row above did not lose any information. In such a case, the current row's character isn't contributing a new match.
- **Action:** Simply move up (`i--`), discarding the current row.

3. Else Case in the First Condition:

- **Interpretation:** If `dp[i][j]` is greater than the left value **and** not equal to the top value, then the only possibility is that the increase came from a diagonal increment (i.e., `dp[i][j] == dp[i-1][j-1] + 1`), which happens only when a match is found.
- **Action:** Prepend `text_02[i]` (the matched character) to the result string, and move diagonally (`i--`, `j--`).

4. `else { j--; }`

- If `dp[i][j]` is not greater than `dp[i][j-1]`, this suggests that the current cell's value comes from the left cell. Thus, move left by decreasing `j`.

Illustration with a Simple Example

Let's use our earlier small example with:

- `text_01 = "ABC"`
- `text_02 = "AC"`

After prepending a space, we have:

- `text_01 = " ABC"`
- `text_02 = " AC"`

Our computed DP table (with indices representing positions in the modified strings) is:

	j=0: ""	j=1: "A"	j=2: "B"	j=3: "C"
i=0: ""	0	0	0	0
i=1: "A"	0	1	1	1
i=2: "C"	0	1	1	2

Note: Here, $m = 2$ (since "AC" has 2 letters) and $n = 3$ ("ABC" has 3 letters).

Now, let's backtrack:

- Start:** Set $i = 2$, $j = 3$, and $str = ""$.
- Iteration 1:**
 - At $dp[2][3] = 2$
 - Check left: $dp[2][2] = 1 \Rightarrow$ Since $2 > 1$, we enter the first `if`.
 - Now check above: $dp[1][3] = 1 \Rightarrow 2$ is not equal to 1 , so we go to the `else` inside.
 - Action:** Add $text_2[2]$ (which is "C" from "AC") to str . Now, $str = "C"$. Move diagonally: set $i = 1$, $j = 2$.
- Iteration 2:**
 - Now at $dp[1][2] = 1$
 - Check left: $dp[1][1] = 1 \Rightarrow$ Since 1 is **not greater** than 1 (they are equal), we fall into the `else` on the outer condition.
 - Action:** Move left: set $j = 1$.
- Iteration 3:**
 - Now at $dp[1][1] = 1$
 - Check left: $dp[1][0] = 0 \Rightarrow 1 > 0$, enter the first `if`.
 - Check above: $dp[0][1] = 0 \Rightarrow 1$ is not equal to 0 , so we add the match.
 - Action:** Add $text_2[1]$ (which is "A" from "AC") to str . Prepending it gives: $str = "A" + "C" = "AC"$. Move diagonally: set $i = 0$, $j = 0$.
- Loop Ends:** Now i or j is 0, so we exit. The resulting LCS is "AC".

This matches our expected result.

Final Explanation

- The backtracking algorithm** starts at $dp[m][n]$ (bottom-right) and moves toward the top-left.

- **When the left neighbor's value is lower** than the current cell, it means the current cell's value was "boosted"—usually because of a match.
 - **But** if the top neighbor has the same value, then that boost didn't come from the current character; it may be carried over from above, so we move up.
 - **Otherwise**, a boost that is not mirrored above indicates a diagonal match; we add the corresponding character to our answer and move diagonally.
- **If the left neighbor is not lower**, then the best computed LCS length came from the left, so we simply move left.

This method cleverly uses comparisons of neighboring cells in the DP table to deduce where the increase in value (i.e., a match) occurred, thereby reconstructing the LCS.